Which Coordination Mechanisms Keep the Record Faithful? Fault-Injection Ablation of a Hub-Orchestrated, LLM-Assisted Interview-Preparation System

Anonymous Author(s)

Anonymous Institution

Abstract. Applications that combine a learned scorer, rules, a small language model and a reinforcement-learning policy are often described as multi-agent systems although one controller coordinates them. We audit one such system, a spoken technical-interview practice pipeline, and ask which orchestrator-level mechanisms keep the stored session record faithful to the authoritative scorer when components fail. Tracing 186 turns shows a centrally orchestrated hub with a measurable write-authority partition: the scorer, validator and feedback roles change no session state, the follow-up role changes only the question queue, and only the hub commits scores. We then ablate seven mechanisms under five registered fault classes, replaying recorded scorer outputs for 64 answers through the real orchestrator code (no live model was available). All 14 pre-registered predictions held. The shipped system committed malformed scorer outputs as scores in every session of the malformed-score condition, and an exploratory count showed difficulty oscillation from the interaction of the policy and the guardrails. A strict hand-off contract removed the silent commits without changing fault-free behaviour, at the price of availability when every call is malformed. With real sockets, a hanging language-model service cost 24.7 s per turn against 0.14 s when healthy; a circuit breaker cut the median to 3.1–6.2 s under modelled candidate pacing (Sect. 5.6). A lexical grounding check accepted truncated paraphrases of unsupported credit. The evidence is replayed, small and says nothing about live language-model quality or learning.

Keywords: orchestration · fault injection · ablation · LLM-integrated systems · authority separation · circuit breaker

1 Introduction

Systems that wrap a small language model (LLM) in a larger pipeline are frequently presented as multi-agent: a scorer, a feedback writer, a follow-up generator and a policy each carry an "agent" label. Whether the label is earned matters for how such systems should be tested. If the components are peers that negotiate, failures arise between agents [1]; if a single controller calls them in a fixed order, the relevant questions are about the controller's contracts with each

component: who may write what, what happens when a component fails or lies, and what waiting costs.

We study an implemented system of the second kind, a spoken technical-interview practice pipeline whose code we audit rather than assume. Its design intent is that a learned scorer is the only authority on correctness, while an LLM only phrases feedback and follow-up questions. We ask:

RQ1 (authority) Which role changes which part of the session state?
RQ2 (necessity) For each orchestrator-level mechanism, is there a registered fault under which removing it violates a registered invariant that the full system satisfies?
RQ3 (cost) What does the failure path cost in calls and time when the LLM service is healthy, refusing or hanging?
RQ4 (grounding) What does the LLM-output grounding check accept and reject, and what can it not see?

Contributions. (i) An execution-path audit that states plainly what is and is not multi-agent in the system and an empirically derived write-authority matrix (Sect. 3, 5.1). (ii) A mechanism-by-fault ablation with pre-registered invariants and predictions, run on the real orchestrator through replay, including cells where the shipped system itself fails (Sect. 5.2). (iii) Two evidence-driven changes with before/after measurements: a strict scorer-to-hub contract (Sect. 5.5) and a circuit breaker for the LLM service (Sect. 5.6). (iv) A measurement of failure-path latency with real sockets (Sect. 5.3) and a probe of the grounding check on constructed outputs (Sect. 5.4).
Not claimed. We make no claim that the system is decentralised, that agents negotiate, that any mechanism improves feedback quality or learning, or about rates with a live LLM: no LLM output was produced in any experiment (Sect. 7).

2 Related Work

Agents and LLM multi-agent systems. Wooldridge and Jennings' weak notion of agency lists autonomy, social ability, reactivity and pro-activeness [11]; we use it to test the label. AutoGen frames applications as conversing agents [12], and MetaGPT assigns roles along a standard-operating-procedure pipeline with verification of intermediate results [3]. The system studied here resembles the latter in its fixed role pipeline but has no inter-agent conversation. *Failures and fault injection.* The MAST taxonomy groups failures of multi-agent LLM systems into specification issues, inter-agent misalignment and task verification [1]. Huang et al. inject faulty agents into LLM collaboration and report that a hierarchical structure is more resilient than two alternatives [4]; MAS-FIRE defines fault types injected by prompt modification, response rewriting and message routing [5]. Our faults act at service boundaries of a pipeline whose authoritative component is not an LLM. *Interview practice systems.* Zara generates practice interviews and feedback with a single LLM and retrieval [13]; PolyInterview runs parallel evaluators and links scores to behavioural evidence [10]; user studies examine

student readiness and trust in automatic feedback [2, 9]. We found no work in the retrieved summaries that ablates coordination mechanisms of such a system under injected faults; this is a statement about those summaries only, and full texts were not read. *Dependability patterns.* The circuit breaker stops calls to a failing dependency [6]. *LLM judges* show position, verbosity and self-enhancement biases [14], one reason the studied design keeps scoring outside the LLM.

3 System and Architecture

Table 1 and Fig. 1 summarise the roles; all code references are to the audited snapshot. A single session object (the hub) owns the session state, serialises every mutation with one lock, and calls the other roles in a fixed order per turn: evaluate, validate, write feedback, commit, adapt difficulty, decide a follow-up, log. The evaluator is a learned scorer (sentence embeddings, a vector index and a fine-tuned cross-encoder) that returns a score and lists of covered, missing and incorrect concepts; it generates no text. A rule-based validator caps the score at 0.65 when a mandatory concept is absent and subtracts a bounded penalty. The feedback and follow-up roles call a small LLM service (Qwen2.5-1.5B, quantised) over HTTP with a 6 s timeout and two attempts; both fall back to deterministic templates, and the feedback text is accepted only if a lexical grounding check passes. The difficulty policy is a frozen PPO [8, 7] policy trained in simulation, followed by rule-based guardrails that may override its action.

Under the weak notion of agency [11], only the difficulty policy is a learned, reactive decision-maker; the follow-up and feedback roles are LLM-assisted functions with fallbacks and the rest are deterministic. No role calls another role, holds private state across turns, negotiates or plans. We therefore describe the system as a *hub-orchestrated pipeline with an authority partition*, not as a decentralised multi-agent system.

4 Method

4.1 Fixture and evidence class

All E1 and E4 runs are *replay*: the real orchestrator code runs with the recorded output of the production scorer for 64 constructed answers (8 questions $\times$ 8 answer types), served through the orchestrator's evaluator hook. Session $k \in \{0..7\}$ asks the 8 questions and gives the k-th recorded answer to each, so the 8 sessions cover each answer once. A follow-up is answered with the recorded answer of its root question; the replay candidate therefore never improves. The timer is fixed so no timing modifier applies, the PPO checkpoint and guardrails are the shipped ones, and the LLM service is "down" (the orchestrator's own fallbacks run). E2 and E5 use *real sockets* to a local server bound to the hard-coded service address with real timeouts. E3 uses *constructed* outputs. No evaluator or LLM was executed.

Table 1. Roles, kinds and whether a role meets the weak notion of agency [11] (autonomy, social ability, reactivity, pro-activeness) as implemented.

Role	Kind	LLM / RL	Independent decision-maker?
Hub (session controller)	rule-based state machine	none	coordinator, not a peer
Scorer	learned scorer, no generation	none (not an LLM)	no: function of its input
Score validator	rule-based caps and penalty	none	no
Feedback role	LLM text with template fallback	LLM (optional)	no: function with fallback
Follow-up role	threshold policy plus LLM text or template	LLM (optional)	partly: a policy invoked by the hub
Difficulty policy	PPO, then rule overrides	RL	yes: the only learned, reactive decision-maker
Timer, selector, sandbox	deterministic modules	none	no

4.2 Variants, faults, invariants

Seven mechanisms are removed one at a time by patching the instance (repository code unchanged): V1 score validator, V2 guardrails, V3 PPO policy (heuristic recovery path), V4 follow-up role, V5 the cap of two consecutive follow-ups, V6 fail-closed scorer hand-off (an outage becomes an ordinary zero-score answer), V7 attempt rollback; V0 is the shipped system. Faults: F1 permanent scorer outage; F2 intermittent outage ($p = 0.25$ per call, seeds 0–19); F3 malformed scores cycling through None, NaN, a string, 1.7 and -0.3; F4 a score of 0.95 for answers whose mandatory gate failed; F5 each answer submitted twice through the retry path. Seven invariants are computed per session: I1 no committed score without a valid scorer return; I2 stored scores finite in $[0, 1]$; I3 mandatory gate respected; I4 the session terminates within 40 driver turns; I5 at most two consecutive follow-ups; I6 consistent state counts; I7 after a score below 0.30 in the policy phase, difficulty does not rise. Predictions for 14 cells were written before the run, labelled "known" (from reading the code) or "unsure". Protocol, scripts and amendments were hashed and tagged before each run; the two later experiments (E4, E5) were registered as amendments after earlier results, which is disclosed here. Intervals are Clopper–Pearson; sessions from different seeds reuse the same 64 answers, so all intervals are descriptive.

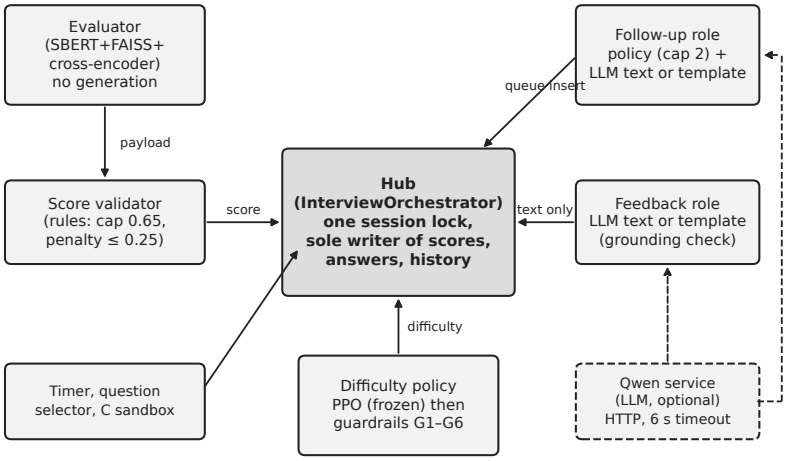


Fig. 1. Hub-orchestrated pipeline. Measured write authority over 186 traced turns: scorer, validator and feedback wrote no state key; the follow-up role changed only the question queue; the difficulty policy changed only difficulty and policy fields.

Table 2. State keys changed per role (traced).

Role	Keys	Examples
Scorer and validator	0	–
Feedback role	0	–
Follow-up role	1	question queue
Difficulty policy	14	difficulty, history, policy status
Hub: commit step	22	scores, raw scores, answers, topic history
Hub: dispatch, report, other	8	question index, status, report fields

5 Results

5.1 Write authority (RQ1)

Table 2 lists how many distinct state keys each role changed over 186 turns (64 primary and 122 follow-up turns in 8 sessions). The follow-up role's only effect on state is the question queue. This does not mean the LLM-facing roles cannot influence scores: the follow-up text is the question text given to the scorer when the follow-up is answered, and follow-up scores are committed and seen by the difficulty policy. We read this indirect path from the code; its size was not measured because replay cannot measure it.

Table 3. Selected E1 cells: sessions violating the invariant / sessions (total violation count). A dash marks cells without a registered prediction; "Predicted" = a registered prediction of a violation (it held).

Fault	Variant	Invariant	Violating sessions	Predicted
F3	V0 shipped	I1 commit without a valid scorer return	160/160 (3796)	yes
F3	V1 no validator	I2 invalid stored numbers	160/160 (1532)	yes
F4	V1 no validator	I3 mandatory gate	8/8 (144)	yes
F1	V6 no fail-closed	I1	8/8 (192)	yes
F2	V6 no fail-closed	I1	160/160 (816)	yes
F0	V5 no follow-up cap	I4 / I5	8/8 / 8/8	yes
F5	V7 no rollback	I6	8/8	yes
F2	V0 shipped	I4 (driver gave up after 3 outages)	15/160	–
F1	V0 shipped	I4 (never reaches the end)	8/8	–
F0	V0 shipped	I7 up after low score	7/8 (39)	–
F0	V2 no guardrails	I7	3/8 (4)	yes
F0	V3 no PPO	I7	0/8 (0)	–

5.2 Mechanism ablation (RQ2)

752 sessions were run in 18 cells. All 14 of 14 predictions held (Table 3); this includes the cells labelled "unsure". Most "known" predictions are, as expected, properties the mechanisms were written to provide; the informative results are the cells where the shipped system fails or where an ablation shows no effect.

Finding 1: malformed scorer output is committed. With F3, the shipped system committed a score in every session although the scorer never returned a valid one: None, NaN and non-numeric values were committed as 0.0 (a zero that the candidate would see as an assessment), -0.3 as 0.0, and 1.7 as 1.0 (or 0.65 when the mandatory cap applied), i.e. full marks from an out-of-contract scorer value. This was predicted from reading the code, so it is a confirmed weakness, not a surprise. *Finding 2: some ablations show where a mechanism is redundant.* Without the validator and with F3, the out-of-range 1.7 reaches the stored raw scores while the committed score is clamped to 1.0 by the timer's own clamp, so the validator is not the only guard for I2 on committed scores; all 160/160 sessions violated I2 through the raw-score list. *Finding 3: the policy and the guardrails interact.* In the fault-free replay the shipped system raised difficulty after a score below 0.30 in 7/8 sessions, versus 3/8 without guardrails and 0/8 without PPO. Guardrail G1 lowers difficulty after a low score only when it lies in a mid band; at the lowest level it does not apply, the PPO action is taken, and the path oscillates (Fig. 2). An exploratory, non-registered count of direction reversals in the policy phase averaged 10.62 per session for the shipped system, 0.5 without guardrails and 0.12 without PPO. Of 150 policy-phase decisions in the shipped system, 74 were taken by a guardrail and 76 by PPO. The replay

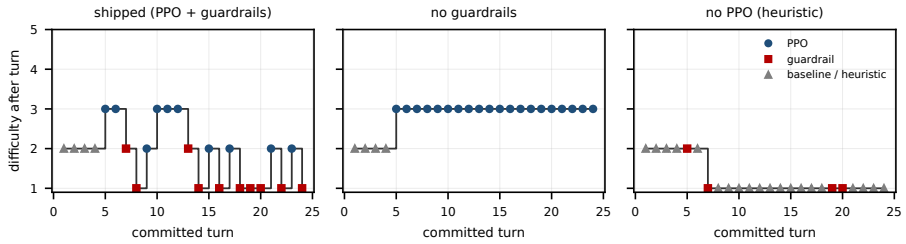


Fig. 2. Difficulty after each committed turn, replay session 0, fault-free.

candidate scores low and never improves, so the size of this effect is specific to the fixture.

5.3 Cost of the failure path (RQ3)

Against a local server at the service address, the median time for the answer-handling call over 24 turns was 0.14 s with a healthy service, 0.69 s when the connection was refused and 24.7 s when the server accepted but never answered (maximum 24.8 s; 495 s in total). Client-side attempts were 40, 80 and 80. The hanging case matches the analytic worst case from the code constants (two attempts of 6 s for feedback and for the follow-up call). A field recording the last service state exists in the feedback role but is never read; there was no circuit breaker. A "healthy" response here is a scripted template response, so the 0.14 s measures orchestration and transport only.

5.4 Grounding check (RQ4)

For the 64 recorded scorer payloads we built constructed LLM outputs by fixed templates and passed them to the service's validator (Table 4). Verbatim contradictions, score leakage and fluff were always rejected. A narrative crediting a truncated paraphrase of a missing concept was accepted in 40 of 57 cases; this is an existence demonstration of the lexical limit, not a rate, because we wrote the paraphrase. The faithful rendering of the scorer's own evidence was rejected in 47 accepted of 64: in 17 payloads the same string is both a covered and a missing concept, an inconsistency of the stored scorer evidence that no component reconciles.

5.5 Strict scorer-to-hub contract (E4)

Finding 1 motivated a change: with a configuration key (default on), a score that is not a finite number in $[0, 1]$ is treated like an outage (fail closed); switching the key off reproduces the earlier behaviour inside the same code. Table 5 compares the two on the same fixture. Fault-free paths (scores, difficulty, follow-ups, decision

Table 4. Constructed outputs accepted by the grounding check / applicable payloads.

Constructed output	Accepted
C0 faithful to scorer evidence	47/64
C1 missing concept listed as correct (verbatim)	0/57
C2 incorrect claim listed as correct (verbatim)	0/10
C3 "correctly explained" + missing concept	0/57
C4 credit to a truncated paraphrase	40/57
C5 score leakage	0/64
C6 generic fluff	0/64

Table 5. E4: invalid commits (all calls malformed, 8 sessions per cell; strict sessions finishing).

Malformation	Baseline commits	Strict commits	Strict sessions finishing
None	192	0	0/8
NaN	192	0	0/8
string	192	0	0/8
1.7	184	0	0/8
−0.3	192	0	0/8

sources) were identical between the variants. When every call is malformed the strict variant commits nothing and no session progresses, which is the integrity-for-availability trade-off. When 10 % of calls are malformed ($F6$, 20 seeds × 8 sessions) the baseline committed an invalid score in 144 of 160 sessions (375 commits) and the strict variant in 0; 160 of 160 strict sessions still finished, with a mean of 23.25 committed answers (baseline 23.26).

5.6 Circuit breaker (E5)

The measurement in Sect. 5.3 showed that a service that accepts connections but never answers costs 24.7 s per turn. We added a circuit breaker (opens after two consecutive failures, cooldown 60 s, one probe when half-open; parameters fixed from the timeout constants before the run) used at the three service call sites; a switch reproduces the earlier behaviour. The breaker's clock was advanced by a pacing P after each turn to model candidate think time, while the HTTP timeouts stayed real (Table 6). The first turn still pays 12.4 s (two attempts to reach the threshold). Afterwards a probe is made once per cooldown window; with $P = 30\,\text{s}$ the probe falls on every second turn, and with $P \geq 60\,\text{s}$ on every turn, so the median per turn is 3.1 s and 6.2 s against 24.7 s without the breaker, and total time over 24 turns falls from 495 s to 80–155 s. A longer pacing ($P = 120\,\text{s}$) does not lower the cost further because the cooldown has already elapsed at every turn (6.2 s). With the healthy service the breaker never opened (0 openings); in the recovery run the first call after the service returned closed it again with 0 turns of delay. Committed scores were identical across all runs, as expected because

Table 6. E5: answer-handling time per turn with a hanging service (24 turns, session 0, real timeouts); client-side attempts.

Condition	Median (s)	Total (s)	Attempts
Hanging, no breaker	24.7	495	80
Hanging, breaker, $P = 30\,\mathrm{s}$	3.1	80	13
Hanging, breaker, $P = 60\,\mathrm{s}$	6.2	155	25
Hanging, breaker, $P = 120\,\mathrm{s}$	6.2	155	25
Refusing, breaker, $P = 60\,\mathrm{s}$	0.17	4	25(vs. 80 without)
Healthy, breaker, $P = 60\,\mathrm{s}$	0.13	3	40

only waiting changes. The cost remains a function of the cooldown parameter, which was not tuned; a longer cooldown trades slower recovery for fewer probes.

6 Discussion

What the architecture buys. The measured authority partition is the property that makes the design testable: because only the hub commits scores and no LLM-facing role writes state, a failure of the LLM service can degrade feedback text and follow-up quality but, in the replay, never the record of scores. The partition is incomplete in one respect that the code shows but replay cannot size: follow-up text influences later scoring and the policy's observations. *What the ablation says.* Most mechanisms did what they were written to do. The useful results were the ones the predictions did not make interesting: the shipped system's silent commit of malformed output, the interaction between two decision components acting on one variable, and the redundancy between the validator and the timer's clamp. *Integrity versus availability.* Failing closed converts silent errors into retries; with every call malformed no session can progress, and with a 10 % malformation rate availability is essentially unchanged. *Is it multi-agent?* In the sense of agents that communicate and negotiate, no. The system is a case for the claim that the vocabulary of multi-agent systems can hide the question that matters for testing: who may write what.

Practical consequences. Four checks follow from the measurements and are cheap for any orchestrated LLM pipeline: (a) state, in code, which role may write which field and test it by tracing; (b) validate the type and range of every value that becomes a committed record and fail closed on violations; (c) give every optional dependency a breaker so that a hung service costs one timeout per cooldown rather than per turn; (d) when two components act on one variable, test their joint behaviour on a stationary input, not only each alone.

7 Limitations and Threats to Validity

(1) No live LLM, scorer or sandbox ran; weights and network were unavailable, so the LLM path was exercised only through its fallbacks and templates. Nothing

here measures feedback quality, coaching value or learning. (2) The fixture is 64 constructed answers to 8 questions; the scorer outputs are recorded; the replay candidate never improves and follow-ups are answered with the root answer, which inflates follow-up counts. (3) Variants are instance patches written by us; V6 re-expresses an earlier defect as an ablation. (4) Predictions for many cells came from reading the same code the ablations test; the "unsure" cells are few. (5) E4 and E5 changes were designed after seeing E1 and E2 results; they were pre-registered before their own runs but not before the study. (6) The grounding probe uses outputs we wrote; the paraphrase result is an existence demonstration. (7) Timing was measured on one 4-core machine with a local server; client-side attempt counts are reported because server-side counts disagreed for reasons we did not investigate. (8) The PPO checkpoint was trained in simulation and is not evaluated here. (9) Scorer, embedding and LLM licences were not checked in this environment. (10) References were verified from search summaries only.

8 Conclusion

A hub-orchestrated LLM-assisted pipeline can be examined through the contracts between the hub and its roles rather than through the label "multi-agent". Tracing, fault-injected mechanism ablation and two small, measured changes showed what keeps the stored record faithful in a replayed setting, where the shipped system fails, and what its failure handling costs. They do not show that the system coaches well or that the results hold with a live model.

Reproducibility. Protocol, scripts, result files and a build script are in an anonymised artefact; every number in the text are generated from the result files. Randomness is limited to seeded fault schedules; the replay is otherwise deterministic. Environment: Python 3.11, 4 CPU cores, no GPU; the repository's pinned dependency versions are not satisfied by this environment, and nothing reported depends on a pin that was verified.

References

1. Cemri, M., Pan, M.Z., Yang, S., Agrawal, L.A., Chopra, B., Tiwari, R., Keutzer, K., Parameswaran, A., Klein, D., Ramchandran, K., Zaharia, M., Gonzalez, J.E., Stoica, I.: Why do multi-agent LLM systems fail? arXiv preprint arXiv:2503.13657 (2025)
2. Gomez, N., Batham, S.S., Volonte, M., Do, T.D.: Virtual interviewers, real results: Exploring AI-driven mock technical interviews on student readiness and confidence. arXiv preprint arXiv:2506.16542 (2025)
3. Hong, S., Zhuge, M., Chen, J., et al.: MetaGPT: Meta programming for a multi-agent collaborative framework. In: International Conference on Learning Representations (ICLR) (2024)
4. Huang, J.T., Zhou, J., Jin, T., Zhou, X., Chen, Z., Wang, W., Yuan, Y., Lyu, M., Sap, M.: On the resilience of LLM-based multi-agent collaboration with faulty agents. In: Proceedings of the 42nd International Conference on Machine Learning, PMLR 267. pp. 26202–26226 (2025)

5. Jia, J., Deng, Z., Chen, Z., Wang, Y., Zheng, Z.: MAS-FIRE: Fault injection and reliability evaluation for LLM-based multi-agent systems. arXiv preprint arXiv:2602.19843 (2026)
6. Nygard, M.T.: Release It! Design and Deploy Production-Ready Software. Pragmatic Bookshelf (2007)
7. Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., Dormann, N.: Stable-Baselines3: Reliable reinforcement learning implementations. Journal of Machine Learning Research **22**(268) (2021)
8. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O.: Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347 (2017)
9. Vaishampayan, S., Brown, C.: Will you trust me more than ChatGPT? evaluating LLM-generated code feedback for mock technical interviews. In: Cooperative and Human Aspects of Software Engineering (CHASE) (2025)
10. Wen, Z., Cao, J., Wang, Z., Chen, C., Liu, X., Yin, J., Li, Z.: PolyInterview: An LLM-based platform for immersive mock interview practice with comprehensive multimodal assessment. arXiv preprint arXiv:2607.10310 (2026)
11. Wooldridge, M., Jennings, N.R.: Intelligent agents: theory and practice. The Knowledge Engineering Review **10**(2) (1995). https://doi.org/10.1017/S0269888900008122
12. Wu, Q., Bansal, G., Zhang, J., Wu, Y., Zhang, S., Zhu, E., Li, B., Jiang, L., Zhang, X., Wang, C.: AutoGen: Enabling next-gen LLM applications via multi-agent conversation. arXiv preprint arXiv:2308.08155 (2023)
13. Yazdani, N., Mahajan, A., Ansari, A.: Zara: An LLM-based candidate interview feedback system. arXiv preprint arXiv:2507.02869 (2025)
14. Zheng, L., Chiang, W.L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E.P., Zhang, H., Gonzalez, J.E., Stoica, I.: Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. arXiv preprint arXiv:2306.05685; NeurIPS 2023 (2023)

A Pre-registered predictions and outcomes

Table 7. All 14 pre-registered E1 predictions. Violation = at least one session violates the invariant. Basis: "known" = from reading the code before the protocol, "unsure" = no prior expectation.

Fault	Variant	Inv.	Basis	Predicted	Observed (sessions)	Match
F1	V6	I1	known	violation	8/8	yes
F2	V6	I1	known	violation	160/160	yes
F3	V0	I1	known	violation	160/160	yes
F3	V1	I2	unsure	violation	160/160	yes
F4	V1	I3	known	violation	8/8	yes
F0	V5	I5	known	violation	8/8	yes
F0	V5	I4	known	violation	8/8	yes
F5	V7	I6	unsure	violation	8/8	yes
F0	V2	I7	unsure	violation	3/8	yes
F0	V4	I5	trivial	none	0/8	yes
F1	V0	I1	known	none	0/8	yes
F2	V0	I1	known	none	0/160	yes
F4	V0	I3	known	none	0/8	yes
F5	V0	I6	known	none	0/8	yes